Circuit-Breaker Thresholds, Worked Out by Hand

A breaker trips after k consecutive failures and cools down before probing. We derive how likely transient noise trips it, and how long a real outage stays contained.

What this document does. A circuit breaker stops a client from hammering a failing server: after k consecutive failures it OPENs (fails fast), waits a cooldown, then HALF-OPENs to probe. This document treats the threshold as a probability question — how often does random transient noise trip the breaker spuriously, and how many calls until a true outage trips it — and works the numbers for a concrete error rate.

Concepts covered: the three-state machine · consecutive-failure threshold · p^k spurious-trip probability · expected calls to a run of k · cooldown containment · choosing k.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: three states, two thresholds

$$\text{CLOSED} \xrightarrow{k \text{ consecutive failures}} \text{OPEN} \xrightarrow{\text{cooldown } t_o} \text{HALF\_OPEN} \xrightarrow{\text{probe ok}} \text{CLOSED}.$$

Parameters: failure threshold $k = 5$ consecutive failures, cooldown $t_o = 30\,\text{s}$. In CLOSED, calls pass through; in OPEN, calls fail *fast* without touching the server; in HALF_OPEN, a single probe decides whether to close or re-open.

> **Intuition**
>
> The breaker's job is to stop wasting calls (and user time) on a server that is clearly down, and to give it room to recover instead of being stampeded. CLOSED is "normal," OPEN is "don't bother, fail instantly," HALF_OPEN is "cautiously test the water." The two thresholds — how many failures trip it, how long it waits — are the whole tuning surface.

2 Step 1 — Spurious trips from transient noise

Suppose the server is actually *healthy* but has a transient error rate $p = 0.3$ (rate limits, blips). The breaker trips only on $k = 5$ *consecutive* failures. Assuming independence, that exact run has probability

$$P(k \text{ in a row}) = p^k = 0.3^5 = 0.00243.$$

So roughly 1 in 412 windows of 5 calls would trip the breaker on noise alone — the threshold filters out almost all transient flapping.

> **Mini-example — this operation in isolation**
>
> Why *consecutive* and not a rate? A simple "$\geq 30\%$ errors" rule trips constantly at $p = 0.3$. Requiring a *run* of 5 raises the bar to 0.3^5 — exponentially stricter in k. Each extra required failure multiplies the spurious rate by p: $k = 3 \to 0.027$, $k = 5 \to 0.0024$, $k = 7 \to 0.00022$. The exponent is your noise filter.

3 Step 2 — Expected calls until a spurious trip

How long, on average, before transient noise produces a run of $k = 5$? The expected number of Bernoulli(p-failure) trials to first see k consecutive failures is

$$\mathbb{E}[\text{calls}] = \frac{1 - p^k}{p^k\,(1 - p)}.$$

With $p = 0.3$, $k = 5$:

$$\mathbb{E} = \frac{1 - 0.00243}{0.00243\,(0.7)} = \frac{0.99757}{0.001701} \approx 586 \text{ calls}.$$

A healthy-but-noisy server trips the breaker about once every 586 calls — rare enough to tolerate, but not never. Raise k to push this out further.

4 Step 3 — A real outage trips immediately

If the server is genuinely down, $p = 1.0$ and the run of k is reached in exactly k calls:

$$\text{calls to trip} = k = 5.$$

After 5 wasted calls the breaker OPENs and every subsequent call fails fast — the client stops sending traffic to a dead server within 5 attempts.

server state	failure rate p	calls to trip
healthy, noisy	0.30	$\sim$ 586 (rare, spurious)
degraded	0.70	$\sim$ 17
down	1.00	5 (exactly k)

5 Step 4 — Cooldown contains the blast

Once OPEN, the breaker rejects calls for $t_o = 30\,\text{s}$ before probing. If the client would otherwise have sent $r = 50$ requests/second, the cooldown spares the failing backend

$$r \cdot t_o = 50 \times 30 = 1{,}500 \text{ requests}$$

it never has to reject — exactly the load that would prevent it recovering. Then a single HALF_OPEN probe tests recovery: success → CLOSED, failure → another t_o of OPEN.

> **Watch out**
>
> Tuning is a trade-off. Small k / short t_o trips eagerly (overreacts to blips, but fails fast); large k / long t_o tolerates noise (but wastes more calls before protecting, and keeps the server cut off longer after it recovers). There is no universal setting — match k to your transient rate p via p^k, and t_o to the backend's realistic recovery time.

6 The argument, at a glance

#	Idea	Formula	Value
1	Spurious trip probability	p^k	0.00243
2	Calls to spurious trip	$\frac{1-p^k}{p^k(1-p)}$	$\sim$ 586
3	Calls to trip (outage)	k	5
4	Requests spared by cooldown	$r\,t_o$	1,500

7 Equation cheat-sheet

States	CLOSED → OPEN → HALF_OPEN → CLOSED
Spurious-trip probability	p^k (k consecutive failures)
Expected calls to a run of k	$(1-p^k)/(p^k(1-p))$
Calls to trip on outage	k (since $p = 1$)
Requests spared by cooldown	$r \cdot t_o$

A Reference implementation

```
def spurious_trip(p, k):
  return p ** k                         # P(k in a row)

def calls_to_trip(p, k):                 # expected, healthy noise
  pk = p ** k
  return (1 - pk) / (pk * (1 - p))

print(round(spurious_trip(0.3, 5), 5))        # 0.00243
print(round(calls_to_trip(0.3, 5)))           # 586
print(50 * 30)                         # 1500 requests spared
```

— end of document —